

Cloud Data Engineer — Interview Prep (Doc 1)

Security-focused JD: ETL/ELT, ingestion, IaC, secure coding, AWS, Python/SQL, data lakes

Q&A + prep focus per area

Tailored to a JD covering: **cloud data engineering (ETL/ELT, ingestion, IaC, pipelines), secure coding & data/application security, AWS data services & high-performant data stores, Python & SQL, and advanced data lakes** — plus communication. Each section has **prep focus**, then **questions with strong answers**.

SECTION 1 — Cloud Data Engineering (ETL/ELT, ingestion, IaC, pipelines)

Prep focus: be able to design a pipeline end to end, explain batch vs streaming ingestion, incremental loading, orchestration, idempotency, and Infrastructure as Code.

Q: ETL vs ELT — difference and when to use each? ETL transforms data *before* loading (good when the target is rigid or you must cleanse/mask first). ELT loads raw data first, then transforms *inside* the warehouse/lake (Snowflake/ Redshift + dbt/Spark) — the modern default because storage is cheap and warehouses are powerful. I use ELT for lake/warehouse analytics and ETL when I must transform or mask in flight.

Q: What data ingestion techniques do you know?

- **Batch:** scheduled bulk loads (files, **COPY**, Glue jobs), full or **incremental**.
- **CDC (Change Data Capture):** capture inserts/updates/deletes from a source DB (**AWS DMS**) with low impact — fresher than nightly dumps.
- **Streaming:** continuous event ingestion (**Kinesis, Kafka, Firehose**) for near-real-time.
- **API/file pulls** for third-party data. Choose by latency need, source type, and volume.

Q: Full load vs incremental load? How do you do incremental? Full reloads everything (simple, expensive at scale). **Incremental** processes only new/changed data using a **watermark** (max timestamp/id), **CDC**, or **Glue job bookmarks**; write to date partitions and use **upsert/MERGE** for updates. Must be **idempotent** so re-runs don't double-count.

Q: How do you orchestrate pipelines? **Airflow/MWAA** or **AWS Step Functions** for DAGs with dependencies, retries, and alerting; **Glue Workflows** for Glue-centric flows; **EventBridge** to trigger on new S3 files. I add data-quality gates that fail the run before bad data moves downstream.

Q: What is Infrastructure as Code and why use it? Defining infra (S3, Glue, IAM, Redshift, networking) in code — **Terraform** or **CloudFormation** — so environments are **repeatable, reviewable, version-controlled**, and consistent across dev/QA/prod. No manual "click-ops," easy rollback, and it plugs into CI/CD.

Q: How do you make a pipeline reliable/idempotent? Immutable raw zone, overwrite-by-partition (or MERGE on a key), dedupe keys, retries with backoff, checkpointing for streaming, and quality checks. Re-running yields the same result.

Q: Walk me through a pipeline you'd design. Land raw in S3 (partitioned by date, immutable) → transform with Glue/Spark to Parquet → catalog with a crawler → load Redshift/Snowflake (or

query via Athena) → serve BI/ML → orchestrate with Step Functions/Airflow (event-triggered) → data-quality tests → monitor with CloudWatch → all as Terraform + CI/CD.

SECTION 2 — Secure Coding, Data & Application Security (the differentiator)

Prep focus: this is where you stand out. Know encryption (at rest/in transit), IAM least privilege, secrets management, PII handling/masking, network isolation, and secure-coding basics (injection, input validation, dependency risks).

Q: How do you secure data at rest and in transit?

- **At rest:** encryption with **AWS KMS** (SSE-KMS on S3, encrypted Redshift/RDS/EBS, encrypted Glue/Snowflake storage). Manage keys + rotation in KMS.
- **In transit: TLS/HTTPS** everywhere; enforce SSL on DB connections; VPC endpoints so traffic doesn't traverse the public internet.

Q: How do you apply least privilege in a data platform? IAM roles with narrowly scoped policies (only the specific S3 prefixes/actions needed), per-service roles (Glue role ≠ user), no wildcards, no long-lived keys — use **roles/temporary credentials (STS)**. In the lake, **Lake Formation** gives table/**column/row-level** grants; in the warehouse, RBAC roles.

Q: How do you handle secrets (DB passwords, API keys)? Never hard-code or commit them. Store in **AWS Secrets Manager** (or SSM Parameter Store, KMS-encrypted); fetch at runtime; enable **rotation**. In code, read from env/secret store; keep secrets out of Git (`.gitignore` , secret scanning).

Q: How do you protect PII / sensitive data? Classify and **mask/tokenize** sensitive columns; **column-level & row-level security** (Lake Formation, Snowflake masking policies, Redshift RLS); encrypt; restrict access by role; **Amazon Macie** to discover PII in S3; retention/deletion policies for compliance (GDPR/CCPA); audit access via **CloudTrail**.

Q: What are secure coding practices you follow?

- **Validate & sanitize all input;** use **parameterized queries / bind variables** to prevent **SQL injection** (never string-concatenate SQL).
- **Least privilege** for the code's identity (scoped IAM role).
- **No secrets in code;** use a secret manager.
- **Dependency hygiene** — pin versions, scan for CVEs (e.g., `pip-audit` , Snyk), avoid untrusted packages.
- **Handle errors safely** — don't leak stack traces/PII in logs; log responsibly.
- **Principle of least data** — only read/store what you need.
- Code review, static analysis (SAST), and secret scanning in CI.

Q: How would you prevent SQL injection in a Python data app? Use **parameterized queries** (e.g., `cursor.execute("... WHERE id = %s", (user_id,))`) or an ORM — never format user input

into the SQL string. Validate/whitelist inputs, and give the DB user least-privilege permissions.

Q: How do you secure the network for data services? Put resources in a **VPC**; use **private subnets** for data stores; **security groups** as least-privilege firewalls; **VPC endpoints** for S3/Glue so traffic stays private; no public S3 access (block public access on); WAF for public apps.

Q: How do you audit and monitor for security? **CloudTrail** (API audit), **CloudWatch** alarms, **GuardDuty** (threat detection), **Macie** (PII), access logs on S3, and least-privilege reviews. Alert on anomalous access and failed auth.

Q: Application security basics (OWASP) relevant to data apps? Injection, broken access control, sensitive-data exposure, security misconfiguration, vulnerable dependencies. Mitigate with input validation, authN/authZ, encryption, secure config (no public buckets/default creds), and dependency scanning.

SECTION 3 — AWS Data Services & High-Performant Data Stores

Prep focus: know each service's job and how to make stores fast/cheap.

Q: Name the key AWS data services and what each is for. **S3** (data lake storage), **Glue** (serverless Spark ETL + Data Catalog + crawlers), **Athena** (serverless SQL on S3), **Redshift** (data warehouse), **Kinesis/Firehose** (streaming), **DMS** (DB migration/CDC), **EMR** (managed Spark/Hadoop), **Lambda** (event compute), **Step Functions** (orchestration), **Lake Formation** (governance), **RDS/DynamoDB** (operational DBs).

Q: How do you build a high-performant data store?

- **Columnar + compressed formats** (Parquet/ORC) so queries scan less.
- **Partitioning** by the column you filter on (usually date) → partition pruning.
- **Right file sizes** (avoid small files; compact to ~128MB+).
- **Redshift:** proper **DISTKEY** (co-locate joins), **SORTKEY** (skip blocks on filters), compression, **COPY** for parallel loads, **VACUUM** / **ANALYZE**, concurrency scaling, materialized views.
- **Athena:** partitioning + Parquet + only needed columns to cut scan cost.
- **Caching**, and choosing the right store for the access pattern (warehouse vs key-value).

Q: Athena vs Redshift — when each? **Athena** = serverless, pay-per-query, best for ad-hoc/occasional S3 queries. **Redshift** = provisioned/serverless warehouse for frequent, complex, high-concurrency BI. Bridge with **Redshift Spectrum** to query S3 from Redshift.

Q: How do you control cost on AWS data stores? Parquet + partitioning (scan less on Athena), right-size/pause Redshift or use Serverless, S3 lifecycle to Glacier for cold data, compact small files, and scan-limit workgroups.

SECTION 4 — Python & SQL

Prep focus: expect a live coding round. Be fluent in Python data handling and SQL joins, aggregations, and **window functions**.

Q (Python): How do you process a large file that doesn't fit in memory? Stream/iterate in chunks (generators, `pandas chunksize`, or read line by line), or use **PySpark** for distributed processing. Avoid loading it all at once.

Q (Python): common data-engineering Python tools? `pandas` / `PySpark` (dataframes), `boto3` (AWS), `requests` (APIs), `sqlalchemy` / `psycopg2` (DB), `pytest` (tests), plus generators, comprehensions, error handling, and type hints for clean code.

Q (Python): write a function to dedupe records keeping the latest. Sort by key + timestamp desc, keep first per key (`pandas drop_duplicates(subset=key)` after sort, or `groupby(key).tail/head`, or a dict keyed by id). In SQL/Spark: `ROW_NUMBER()` partitioned by key ordered by ts desc, keep rn=1.

Q (SQL): find the 2nd highest salary / top-N per group. Use `ROW_NUMBER()` / `DENSE_RANK()` over `PARTITION BY group ORDER BY value DESC`, filter rank.

```
SELECT * FROM (
  SELECT e.*, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) rn
  FROM employees e) t WHERE rn = 2;
```

Q (SQL): explain window functions with an example. They compute across a set of rows related to the current row without collapsing them: running totals (`SUM() OVER (ORDER BY ...)`), ranking (`ROW_NUMBER/RANK`), previous value (`LAG`). Great for top-N-per-group, running totals, and period-over-period.

Q (SQL): INNER vs LEFT JOIN; how to find rows with no match? INNER = only matches; LEFT keeps all left rows (NULLs where no match). Find "missing": `LEFT JOIN ... WHERE right.key IS NULL` (anti-join).

Q (SQL): how do you optimize a slow query? Add indexes on filter/join columns, avoid `SELECT *`, filter early, check the execution plan (`EXPLAIN`), reduce data scanned (partitions/Parquet), refresh stats, and avoid unnecessary functions on indexed columns.

SECTION 5 — Building & Managing Data Lakes (advanced)

Prep focus: zones, formats, partitioning, catalog, governance, table formats (Iceberg/ Hudi/Delta), and lake maintenance.

Q: How do you design a data lake? On **S3** with **medallion zones**: **raw** (immutable, exactly as ingested) → **clean/silver** (validated, typed) → **curated/gold** (modeled/aggregated). Store **Parquet**,

partition by date, catalog with **Glue Data Catalog**, govern with **Lake Formation**, and always keep raw so you can reprocess.

Q: How do you handle updates/deletes in a lake (which is append-only)? Use an open **table format** — **Apache Iceberg / Hudi / Delta Lake** — which add ACID transactions, upserts/merge, and time travel on S3. Otherwise, staging + overwrite-by-partition.

Q: How do you keep a lake performant and clean? Partition sensibly (not too granular), **compact small files**, use columnar formats, manage schema evolution, expire old partitions/versions, and add lifecycle rules (Glacier for cold).

Q: How do you govern a data lake (advanced)? **Lake Formation** for centralized, fine-grained (database/table/column/row) permissions and tag-based access; catalog as single source of truth; PII masking; CloudTrail auditing; cross-account sharing / data mesh for multiple teams.

Q: Data lake vs data warehouse vs lakehouse? Lake = cheap, any-format storage (S3). Warehouse = structured, fast SQL (Redshift/Snowflake). **Lakehouse** = warehouse features (ACID, performance) on the lake via Iceberg/Delta — one platform for both.

SECTION 6 — Communication & Behavioral

Prep focus: they explicitly want strong communication. Practice explaining technical work simply and telling STAR stories.

- **"Explain a complex pipeline to a non-technical stakeholder."** Use an analogy, focus on the business outcome, avoid jargon. (Practice this out loud.)
- **"Tell me about a hard problem you solved."** STAR: Situation, Task, Action, Result — include the metric and what you learned.
- **"A time you improved performance / cost."** Concrete before/after.
- **"How do you handle a production data incident?"** Scope → check logs/metrics → isolate → fix → add a preventive check/alert → communicate status to stakeholders.
- **"Disagreement with a teammate?"** Show you listen, use data, and align on the goal.

Communication tips: structure answers (point → example → result), pause and think, ask clarifying questions on design problems, and always tie technical choices back to **business value, cost, and reliability**.

QUICK PREP CHECKLIST

- [] Draw an end-to-end pipeline (ingest → lake → transform → serve) from memory.
- [] Nail the **security** answers — encryption, IAM least privilege, secrets, PII masking, SQL-injection prevention (this JD emphasizes it — most candidates are weak here).
- [] Be fluent in **SQL window functions** and a couple of **Python** data snippets.
- [] Know **Parquet + partitioning** and Redshift **DIST/SORT keys** cold.

- ☐ Explain **Iceberg/Delta/Hudi** for lake updates.
- ☐ Have 3 **STAR stories** ready and practice explaining one pipeline **simply**.
- ☐ Prepare smart questions to ask (their stack, security posture, biggest data pain point).